Using OpsGate

Why, where, and how to use it

Version 7.1 | August 2026

A short reference for why OpsGate exists, where it applies, and how to work with it — written to be useful whether you're deciding to connect a project or actually doing the implementation work.

Why use OpsGate

- **AI coding assistants are capable, but unsupervised they drift.** — They can wander outside the task they were given, touch files they were never meant to (credentials, deployment config, database structure), or push ahead on a judgment call that was really a person's to make.
- **OpsGate puts a consistent, automatic check in front of every AI-assisted change.** — The same rules are applied the same way every time, regardless of which assistant is doing the work or how routine the task looks — nobody has to remember to enforce them by hand.
- **It draws a hard line around what should never be touched** — and a clear line around what needs an explicit go-ahead — deletions, schema changes, permissions, deployment config — before an assistant is allowed to proceed.
- **It only pulls a person in when a decision genuinely can't be made automatically** — an unknown next step, two equally valid options, or work that would quietly expand beyond what was agreed. Everything else proceeds without interruption.

Where it applies

- **Any project where Claude plans and a Replit Agent implements.** — OpsGate sits between the two: Claude checks a request against OpsGate's rules and compiles it into a scoped task; the Replit Agent executes that task inside the real project and reports back.
- **One registration per project.** — A project (a “tenant”) is connected once — given an identity, a credential, and a definition of its normal working area versus what's permanently off-limits. Every AI-assisted change to that project goes through OpsGate automatically from then on.

- **Two connection points, same rules.** — Claude connects on one path, Replit on another — both enforce the identical rule set, so the two assistants can never drift out of agreement with each other.
- **It governs how the work happens, not what gets built.** — Product and scope decisions stay entirely with your team; OpsGate has no opinion on direction.

How to use it

- **Describe what you want in plain language.** — There's nothing special to learn — hand Claude the request the way you normally would.
- **Claude checks it before anything happens.** — It turns the request into a well-formed task, confirms the scope and files involved are actually in bounds, and — for anything higher-stakes — confirms it has the authorization it needs.
- **The Replit Agent does the implementation and reports back.** — What it changed, what it verified, and how — re-checking the same rules along the way.
- **Most of the time, that's the whole interaction.** — A well-scoped, ordinary change goes through without any extra step — it feels exactly like working without OpsGate in place.
- **Occasionally, you'll be asked an actual question.** — Not a rubber-stamp approval, but a real decision point the assistant can't resolve on its own, with the concrete options and their consequences already laid out. Reply with your decision, and the work resumes exactly where it left off.